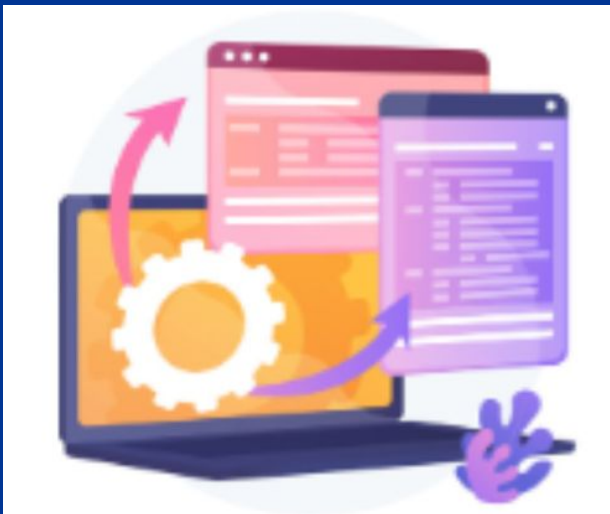




Tema 13. Consultas y transacciones distribuidas

Transacciones distribuidas

Distribución de la información

Las transacciones distribuidas son aquellas que distribuyen los datos en las BDD.

Implica a varios nodos en una red, denominados **agentes**, que se comunican con mensajes de red. Deben cumplir las propiedades ACID (**atomicidad**, **consistencia**, **aislamiento** y **durabilidad**).

Para ello, habrá un **agente raíz**, que será quien inicie la transacción (BEGIN TRANSACTION) desde el **sitio origen**, y sea quien realice el COMMIT/ROLLBACK.



Transacciones distribuidas

Distribución de la información



Transacciones distribuidas

Recuperación de transacciones distribuidas

Para recuperar una transacción distribuida, cada sitio tiene un **manejador de transacción local (MTL)**. El manejador debe:

- **Asegurar la atomicidad** de su transacción.
- Almacenar y actualizar las órdenes de la transacción.
- Para garantizar que la transacción se ejecute correctamente, se debe cumplir:
 - En cada sitio **todas las instrucciones se ejecutan**.
 - Todos los sitios han de tomar la **misma decisión** sobre el COMMIT o el ROLLBACK de la transición global.



Transacciones distribuidas

Protocolos de COMMIT atómicos

Para completar una transacción distribuida el servidor coordinador de la orden de COMMIT o ROLLBACK a todos los nodos participantes en la transacción.

Se suelen usar los protocolos COMMIT en dos fases (2PC) o en tres fases (3PC).

Las fases del protocolo 2PC son:

1. **Votación:** el coordinador consulta a los participantes si se debe hacer el COMMIT
2. **Conclusión:** el coordinador da la orden (COMMIT si todos están de acuerdo)



Consultas distribuidas

Metodología del procesamiento de consultas

Cuando se lanza una consulta, el **procesador de consultas** se encarga de traducirla de **lenguaje SQL** (de alto nivel) a **lenguaje de bajo nivel**.

En un **sistema centralizado**, el coste de ejecución de una consulta se rige por el **número de accesos a disco**.

En un **sistema distribuido**, el coste depende de varios criterios:

- El coste de la **transmisión de datos por la red**.
- **Repetición** (réplica) y **fragmentación**.
- **Procesamiento de intersección simple** (cruce de tablas).



Consultas distribuidas

Repetición y fragmentación

En una BDD, cuando se consultan datos de una tabla, ésta puede estar **repetida** (replicada), **fragmentada**, o **ambas** simultáneamente.

En el caso de **tablas repetidas**, se debe decidir cuál utilizar, que será aquella que tenga **menos costo de transmisión**.

En el caso de **tablas fragmentadas**, se deben realizar **productos** o **uniones** para reconstruir la relación completa. La cantidad de posibles estrategias a seguir para la reconstrucción de los datos puede ser alta, y elegir la adecuada es un proceso complejo.



Consultas distribuidas

Procesamiento de intersección simple

Para realizar una **consulta sobre tablas** que están **distribuidas** (no repetidas) en diferentes nodos, en la que se debe realizar una **intersección** (JOIN), hay varias opciones:

- ★ Enviar copias de todas las tablas al nodo en el que se ejecute la consulta.
- ★ Mandar copia de una tabla al nodo en el que se encuentra otra, cruzarlas allí, y mandar el resultado al siguiente nodo para cruzarlo con la siguiente tabla, y así sucesivamente. El resultado final se envía al nodo en el que se ejecuta la consulta.



Consultas distribuidas

Procesamiento de intersección simple

No hay una estrategia óptima en todos los casos. Hay que tener en cuenta:

- Magnitud de los datos a transferir.
- Coste de transmisión de información entre dos nodos.
- Eficiencia de cálculo de cada nodo.



Consultas distribuidas

Estrategias de procesamiento de consultas

Para ahorrar recursos del sistema al procesar consultas contamos con las siguientes estrategias:

- ❑ **Árboles de consulta:** estructuras de datos que representan el álgebra relacional para agilizar las consultas.
- ❑ **JOIN** (con todas sus variantes)
- ❑ **Transformaciones equivalentes:** se transforman las consultas a sus distintas representaciones en álgebra relacional para decidir la menos costosa.



Bloqueo y concurrencia en transacciones distribuidas

Para realizar el control de las transacciones distribuidas, se pueden emplear:

- ★ Algoritmos basados en la **exclusión mutua sobre datos compartidos**: realizar un **esquema de bloqueo** (varias transacciones leyendo pero sólo una escribiendo).
- ★ Algoritmos que **ordenan la ejecución de transacciones** según un protocolo: los efectos sobre los datos compartidos deben ser equivalentes de forma secuencial (como si las transacciones se realizasen una detrás de otra).



Bloqueo y concurrencia en transacciones distribuidas

Clasificamos los algoritmos de control de transacciones distribuidas como:

- ❖ **Pesimistas**: **realizan bloqueos** al iniciarse las transacciones. Pueden provocar mayores tiempos de espera (peor rendimiento), pero garantizan mayor consistencia de los datos y menos posibilidad de conflictos entre transacciones.
- ❖ **Optimistas**: no realizan bloqueos al iniciar las transacciones, sino que **resuelven posibles conflictos al terminar la ejecución**. Admiten mayor concurrencia y disminuyen los tiempos de espera, pero puede peligrar la consistencia, teniendo incluso que revertir cambios en ocasiones.



Σavante